Notes:

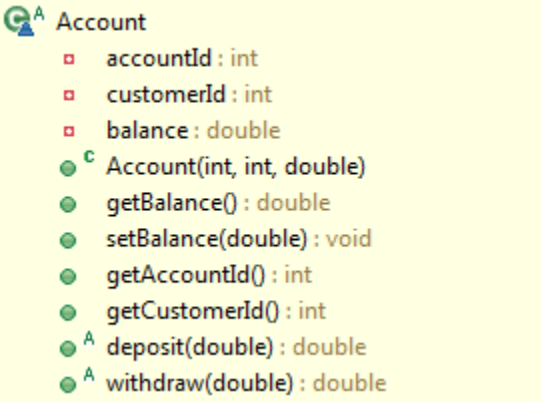
- Go through the problem statement clearly.
- Time limit is 150 minutes.
- Make sure that project is created in eclipse only.
- Create all your java files in package **com** within src folder of eclipse project.
- Make sure that exact class outline is followed as you did in previous assignments.
- You need to zip the eclipse project folder and upload the same in LMS once completed. The project folder will be available in your workspace folder.
- It is mandatory to upload eclipse project and not only java files for your code to be assessed.
- Make sure that there is no compilation error in your code before submission. Even if there is minor error, entire solution could be rejected.
- You may refer previous assignments, course content and internet for any reference.
- Last section is optional.

A banking system is looking for automated solution for managing accounts and related deposit and withdrawal. The system would currently support feature of Savings Account, however, it may have more account types in future. Develop a Java project which serves this system following below guidelines.

Class outline is shared after the guidelines. Ensure the same is met 100%, else your solution would be considered null and void.

Create project BankingSystem in eclipse. Create package **com** within src folder. All classes should be created inside this package.

Create abstract class – Account with attributes account id, customer id and **abstract methods** – deposit and withdraw. Refer below class outline for this class:



Create child class – SavingAccount from this abstract class. This class will implement withdraw and deposit methods where amount to be deposited should be added to the balance and amount to be withdrawn should be deducted from the balance. Withdrawal will not check for zero balance, hence balance can go in negative range. Refer below outline for this class.

```
SavingAccount
    SavingAccount(int, int, double)
    deposit(double) : double
    withdraw(double) : double
```

Create class Bank which has ArrayList of Account. Initialize this list in the constructor.

Write below methods in Bank class:

addAccount – This method will take Account object as a parameter. It will add this object to the list if account with specific id does not exist and return true. If account with specific id already exists, it will return false.

deposit – This method will take account id and deposit amount. Amount will be deposited into specific account with that id. Method will return new balance if account with that id exists, else returns -1.

withdraw – This method will take account id and withdraw amount. It will return -1 if account with specific id does not exist. Else, it will check if withdraw amount is less than or equal to existing balance. If yes, perform withdrawal and return new balance. However, if withdraw amount is greater than existing balance, then exception of class "OutOfBalanceException" should be thrown. This class is explained later.

getUniqueCustomerIds() – This method will return a Set of Integer types, where it will contain unique customer ids across all account objects.

getAccountHashMap – This method will return HashMap with account id as key and respective account object as value.

Refer below outline for this class.

```
Bank
    accounts : ArrayList<Account>
    Bank()
    addAccount(Account) : boolean
    deposit(int, double) : double
    withdraw(int, double) : double
    getUniqueCustomerIds() : Set<Integer>
    getAccountHashMap() : HashMap<Integer, Account>
```

Declare class OutOfBlanceException which extends Exception class.

Constructor of this class will take existing balance as first parameter and withdrawal amount as second parameter. getMessage() method should be overridden such that it returns the difference of balance – withdraw amount as a String. Outline of this class is as below:

```
OutOfBalanceException
  currentBalance : double
  withDrawAmount : double
  OutOfBalanceException(double, double)
  getMessage() : String
```

Write a method in Bank class which returns set of unique customer ids.

Write a method in Bank class which returns Hash Map of account id as key and respective account object as value.

Optional:

Declare class NewBank which has tree set of Account objects. Write method to add account object to this set and returns size of the set after adding the object. Tree set of accounts should maintain accounts in ascending order of account id.(Hint you would need to implement comparable in Account class)

Refer class outline as below:

```
NewBank
  accounts : TreeSet<Account>
  NewBank()
  getAccounts() : TreeSet<Account>
  addAccount(Account) : int
```

Declare a class BankDemo which has main method. Test execution as below:

- Declare Bank object.
- Declare three Account objects with separate account ids(Saving account objects referred by Account references)
- Add these objects in Bank object.
- Test deposit method of bank object
- Test withdraw method of bank object along with the case where OutOfBalanceException is thrown.
- Test remaining methods of Bank object.
- Create fourth account object with same account id matching account objects created before.
- Try adding this account to Bank object.
- If optional part is implemented, then create object of NewBank class and test execution of add account method by adding account objects in different order of account ids.